Team Roles and How It Merges

SOAIDEATHON-S28 - 6 people - 36 hours

This reconciles the original roles guide with how the repo is actually set up. Where they differ, this file wins.

The one rule

Agree the shape of the data before anyone builds. That shape is `docs/api-contract.md`. Frontend builds against a fake version of a response, backend builds the real one, and they meet without conflict because neither ever changed the shape.

The contract is **frozen at hour 4**. After that, you change the contract file first, announce it, then change code.

The six roles

1. Frontend - Student Dashboard

Owns: login, course summary, gap check, lesson cards, practice, quiz, misconception confirm, mastery tracker.

Files: `frontend/src/pages/student/`

Done means: every screen calls the shape in the contract and renders all of its states, including the two refusal states. Real data can come later.

Blocks nobody. Needs the contract only.

2. Frontend - Teacher Dashboard

Owns: misconception heatmap, reasoning paths, gap map, uncertainty flags, before/after, reteach approval, verification queue.

Files: `frontend/src/pages/teacher/`

Done means: same, plus every list has an empty state. Most teacher views start empty and fill during the demo.

Blocks nobody.

3. Frontend - Management Dashboard

Owns: curriculum upload, course and prerequisite structure, audit log, and the auth screens (login, signup, forgot password).

Files: `frontend/src/pages/admin/`, `frontend/src/pages/auth/`

Done means: same. Forgot password is UI only and must not make a network call.

Blocks nobody.

4. Backend + AI engine (Sushree)

Owns: auth, database models, ingestion and chunking, course and prerequisite structure, quiz engine, gap detection, seeding, every HTTP endpoint - **and** retrieval, the alignment score, the refusal rule, the graded-work guardrail, the misconception matcher, prompts, and the provider layer.

Files: all of `backend/`, all of `prompts/`

Done means: the endpoint matches the contract and returns real data from the database, and each "smart" function returns a structured object with a confidence score attached rather than a paragraph of prose.

Frontend does not wait on you. They build against fakes with the same shape.

On load: this is roughly 17 of the 32 features with five people downstream, which is a real bottleneck and a deliberate choice. Two things keep it manageable: the API contract is written first so nobody waits on working code, and roles 5 and 6 absorb everything that does not need Python. If this role falls behind, that is the slack to pull from.

5. Content, language and accessibility

Owns: the source PDFs that become the knowledge base, the diagnostic questions, the misconception list, seed data, translations, and the accessibility toggles.

Files: `backend/data/`, seed content, `frontend` i18n and a11y work

Done means: a demo corpus exists, the diagnostic produces a **predictable** gap, and there are 8-10 **specific** misconceptions each paired with the wrong answer that signals it.

This role is easy to underestimate and it decides whether the demo lands. "Struggles with forces" is worthless. "Treats constant velocity as implying a net force" is a diagnosis a physics teacher recognises instantly.

6. Integration and demo lead

Owns: wiring real endpoints to real frontend calls, seed data for everything not fully live, `docs/demo-script.md`, rehearsal, and the deck.

Done means: the golden path runs start to finish with nobody standing behind a laptop explaining what would happen.

Check in with every pair throughout. Do not parachute in at hour 30.

Two jobs that get forgotten

Both are in the problem statement, so both are scored. They now have owners - keep it that way.

- **Multilingual and accessibility** (`i18n-001`, `a11y-001`) - role 5. Read-aloud, font size, contrast, language switch. Roughly an hour of work for a whole judging criterion.
- **The deck and the pitch** - role 6. SIH weighs presentation heavily and teams routinely budget nothing for it.

Merge order

1. **Skeleton and contracts.** Repo structure, design tokens, database models, API contract - merged to main together, before anyone splits off. *(Done: `cc24ce1`, `c2f7760`.)*
2. **Parallel build.** Everyone on their own branch, in their own folder, merging small and often. Not one giant merge at the end.
3. **Real-logic swap.** Backend replaces canned responses with real ones. The shape never changed, so frontend touches nothing.
4. **Integration and demo.** Golden path fully live, everything else seeded, rehearsed twice.

Branch and commit rules

- Branch per feature: `feat/<feature-id>-<slug>`, e.g. `feat/student-003-gap-lesson`.
- Commit small and often. Push at least every hour so nobody's work is trapped on one laptop.
- `main` must always be green: `./init.sh` passes before you merge.

- If you touch a file you do not own, say so in the team channel first.
- Never commit `.env`, `node_modules/`, or `.venv/`.

What "done" means here

The roles guide says done is "renders whatever comes back". That is right for stage 2 only. For a feature to be marked `passing` in `feature_list.json`, all of this must be true:

1. The `verification` steps for that feature were run exactly as written.
2. The actual output is saved under `evidence/<feature-id>/`. The real terminal text or a screenshot, not a description of it.
3. `./init.sh` still passes.
4. It is committed.

If you cannot produce the evidence, it is not passing. Only one feature may be `in_progress` at a time.

The trap that will cost you the demo

Every tutor response has an `outcome` field with three possible values:

- `answered` - normal explanation with citations and an alignment score
- `insufficient_evidence` - the tutor refused because the curriculum does not cover it
- `graded_work_refused` - the tutor refused because it is graded work, and returned hints

All three arrive as HTTP 200, because a refusal is a **correct** response, not an error. If a screen only handles `answered`, the two features the judges care most about render as blank cards. Handle all three or the build looks broken exactly when it is working.